Parivahan RAG Assistant

An Advanced Hybrid Retrieval-Augmented Generation System
for Indian Motor Vehicle Services

Main Interface — Parivahan RAG Assistant

Assignment: Theme T10.5: Government-Services RAG Chatbot
Variant: Driving Licence & RC (Parivahan / Sarathi)
Live Demo: parivahan-rag-assistant.streamlit.app
GitHub: github.com/JagadishReddyK7/ParivanRAGAssistant

Team Members

2025201006 Jagadish Kollu
2025201027 Erva Rishitha
2025201002 Mannem Tejaswini

Knowledge Base: 24 official PDFs from `parivahan.gov.in` | **LLM:** Gemini 2.0 Flash / Groq Llama-3 | **Embeddings:** `all-MiniLM-L6-v2`

1. Abstract

The Parivahan Sewa platform hosts dozens of dense PDF manuals covering driving licences, vehicle registration, fees, and regulations. For the average citizen, finding a specific form number or fee value across hundreds of pages of bureaucratic text is deeply frustrating. This report describes the **Parivahan RAG Assistant**, an intelligent chatbot that answers questions about Indian motor vehicle services by retrieving from a curated set of 24 official government PDFs.

The system implements a **Hybrid Retrieval-Augmented Generation (RAG)** pipeline that combines dense semantic search (FAISS + MiniLM embeddings) with lexical keyword search (BM25), fusing results via Reciprocal Rank Fusion (RRF). Answers are generated by state-of-the-art LLMs via free-tier REST APIs (Gemini 2.0 Flash / Groq Llama-3). The application is deployed on Streamlit Community Cloud and consistently outperforms pure semantic search on government-specific queries involving exact form numbers, acronyms, and fee amounts.

2. Introduction

2.1 The Bureaucratic Challenge

Citizens interacting with Regional Transport Offices (RTOs) require precise, step-by-step instructions. A user asking *"How do I renew my driving licence after expiry?"* does not want a link to a 40-page PDF — they need the specific form to fill (e.g., Form 9), whether a medical certificate is required, and the exact late fees applicable. Government documentation scatters this information across multiple PDFs: fees in one document, forms in another, FAQs in a third.

2.2 The Problem with Standalone LLMs

Deploying a standalone LLM carries a significant risk: **hallucination**. LLMs rely on parametric memory baked in during pre-training. Government regulations change frequently and include precise identifiers (form numbers, fee amounts, portal URLs) that LLMs routinely confabulate. In a civic tech context, giving a citizen an incorrect legal procedure is potentially harmful.

2.3 The Proposed Solution: RAG

Retrieval-Augmented Generation (RAG) changes the paradigm from *"rely on the model's memory"* to *"retrieve from a verified knowledge base, then generate."* The query lifecycle in our system:

1. **Vectorize:** Convert the user's query into a 384-dimensional embedding vector using a pre-trained sentence transformer.
2. **Retrieve (Dual):** Search a local database of government PDF chunks simultaneously using semantic (FAISS) and keyword (BM25) matching.
3. **Fuse:** Merge semantic and lexical rankings via Reciprocal Rank Fusion (RRF), discarding raw score scales entirely.
4. **Augment:** Inject the top-5 retrieved paragraphs into the LLM's context window as grounding evidence.
5. **Generate:** Instruct the LLM to answer *only* from the injected context, citing PDF name and page number.

2.4 Core Objectives

- **Zero Hallucination:** Prompt engineering prevents the LLM from inventing forms, fees, or URLs absent from retrieved context.
- **Fast Retrieval:** FAISS C++ bindings traverse thousands of chunks in under 5 ms on standard CPU hardware.
- **Exact-Match Safety:** BM25 lexical search ensures precise retrieval of form numbers, acronyms, and fee values.

- **Zero-Cost Deployment:** Free-tier embeddings, free LLM APIs, and Streamlit Community Cloud enable indefinite hosting.
- **Multilingual Output:** Optional Hindi-language response toggle for non-English-speaking citizens.

3. Data

3.1 Knowledge Base: 24 Official Parivahan PDFs

All documents were curated manually from parivahan.gov.in and stored locally in `data/pdfs/`:

#	Filename	Content
1	`Addition_Of_Class.pdf`	Adding a new vehicle class (e.g., LMV) to an existing licence
2	`Change_Of_Address.pdf`	Migrating a licence or RC to a new RTO jurisdiction
3	`Diplomatic_Vehicles.pdf`	Registration protocols for embassy/consulate vehicles
4	`Duplicate_License.pdf`	Lost, stolen, or mutilated licence replacement
5	`Duplicate_RC.pdf`	Retrieving a lost Registration Certificate
6	`Fees_and_User_Charges.pdf`	Master ledger of statutory fees for every service
7	`Frequently_Asked_Questions.pdf`	30+ page FAQ covering edge cases and common queries
8	`HP_Endorsement.pdf`	Hypothecation protocols for bank-financed vehicles
9	`HP_Termination.pdf`	Removing hypothecation after loan clearance
10	`International_Driving_Permit.pdf`	IDP application rules, visa requirements, validity
11	`Issue_Of_Duplicate_Trade_Cert.pdf`	Dealer-specific trade certificate guidelines
12	`Learners_License.pdf`	Requirements, age limits, and tests for beginners
13	`Licensing_Related_Fees.pdf`	Detailed breakdown of LL, DL, and test fees
14	`No_Objection_Certificate.pdf`	Inter-state transfer requirements (NOC)
15	`Permanent_License.pdf`	Transitioning from Learner's to Permanent DL
16	`Permanent_Registration.pdf`	Showroom-to-RTO registration flow for new vehicles
17	`question_bank.pdf`	Traffic sign explanations and legal rules (largest file)
18	`Reassignment_Of_Registration.pdf`	Number plate reassignment rules
19	`Registration_Display.pdf`	Legal dimensions and colour codes for number plates
20	`Renewal_of_RC.pdf`	15-year validity renewals for private vehicles
21	`Renewal.pdf`	Driving licence renewal procedures
22	`Temporary_Registration.pdf`	30-day transit registration for out-of-state purchases
23	`Trade_Certificate.pdf`	Showroom and manufacturer testing regulations
24	`Transfer_Of_Ownership.pdf`	Buying/selling used vehicles and death-of-owner protocols

Table 1: The 24 official Parivahan PDFs forming the knowledge base.

3.2 Data Pipeline and Chunking

PDF text was extracted using `PyMuPDF` (`fitz`), page-by-page, preserving page numbers for citation. Whitespace was normalised using regex. A **Sliding Window Chunking** strategy was applied with the following parameters: chunk size = 500 words, overlap = 50 words, minimum chunk length = 80 characters. The 50-word overlap ensures semantic continuity across chunk boundaries so that answers spanning page breaks are not fragmented.

3.2.1 The Flowchart Exclusion Decision

Many government PDFs contain process flowcharts rendered as vector graphics. PyMuPDF processes these objects linearly by coordinate order, ignoring arrow direction logic, producing nonsensical word salad such

as: *"Click Yes on confirmation box Enter Vehicle Registration Crosscheck details Make Online Payment Pending Success Fail."* Accurately parsing flowcharts requires Multimodal RAG (Vision-Language Models), which was considered out of scope. All detected flowchart fragments were filtered from the chunk corpus.

4. Method

4.1 System Architecture Overview

The system has three fully decoupled pipelines: an **offline Data Extraction Pipeline**, an **offline Hybrid Indexing Pipeline**, and an **online Retrieval & Generation Pipeline**. The offline stages are pre-computed locally and committed to the repository as binary artefacts, so that the deployed Streamlit app never re-processes PDFs.

Figure 1 shows the complete end-to-end architecture.

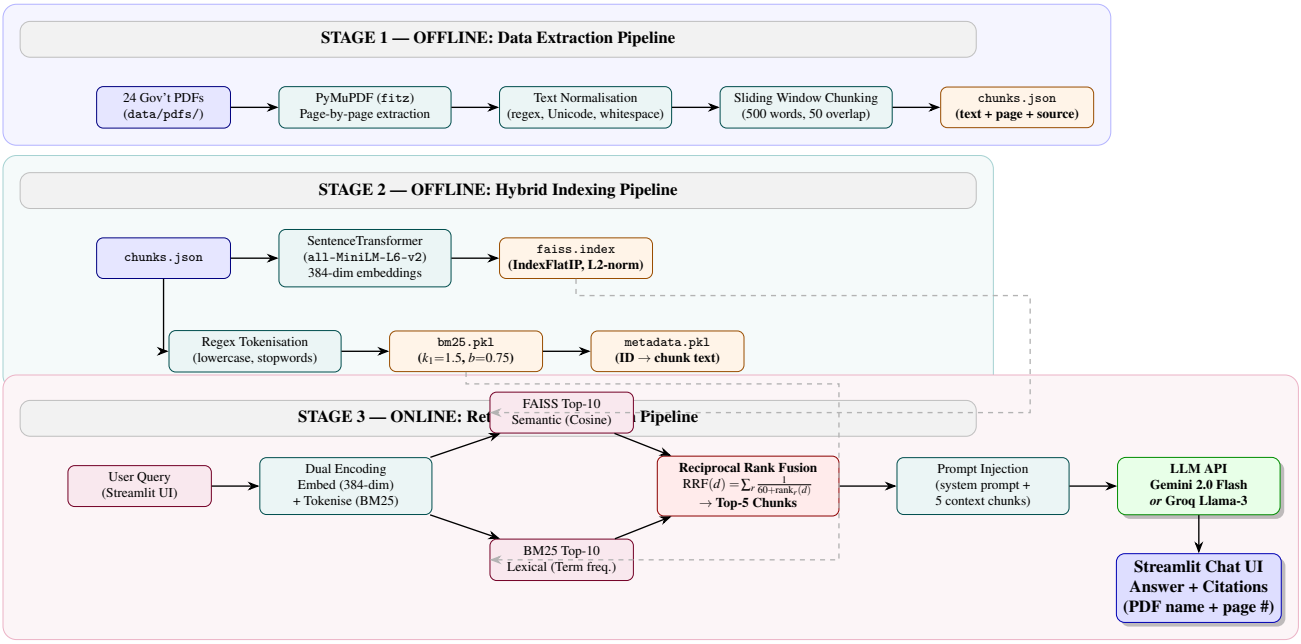


Figure 1: Complete end-to-end architecture of the Parivahan RAG Assistant. Offline stages (blue, teal) are pre-computed and stored as binary artefacts. The online pipeline (purple) loads them at boot and serves queries in real time. Dashed arrows denote binary artefact loading at application startup.

4.2 Dense Retrieval: FAISS + MiniLM

Model. `sentence-transformers/all-MiniLM-L6-v2` — 80 MB, runs entirely on CPU, no GPU required.

Dimensionality. Each chunk and query is embedded into $\mathbb{R}^{384}$.

Index type. `faiss-cpu` with `IndexFlatIP` (Inner Product). Since embeddings are L2-normalised ($\|A\| = \|B\| = 1$), inner product equals cosine similarity:

$$\text{Cosine}(A,B) = \frac{A \cdot B}{\|A\|\,\|B\|} = \sum_{i=1}^{384} A_i B_i \tag{1}$$

FAISS traverses the entire index in under 5 ms on standard laptop hardware.

4.3 Lexical Retrieval: BM25

Dense embeddings fail on out-of-vocabulary (OOV) terms. Querying *"What is Form 20?"* returns Form 21 and Form 4 with nearly identical cosine similarity (≈0.82) — legally a critical error. BM25 scores by exact term

frequency with document-length normalisation:

$$\text{BM25}(Q,d) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i,d)\,(k_1+1)}{f(q_i,d) + k_1\left(1 - b + b\,\dfrac{|d|}{\text{avgdl}}\right)} \tag{2}$$

Standard hyperparameters were used: $k_1 = 1.5$, $b = 0.75$.

4.4 Reciprocal Rank Fusion (RRF)

FAISS scores lie in $[-1,1]$ (cosine similarity); BM25 scores are arbitrary positive floats — the two scales are incompatible for direct summation. RRF solves this by discarding raw scores entirely and combining only the *rank positions*:

$$\text{RRF}(d) = \sum_{r \in \{\text{FAISS},\,\text{BM25}\}} \frac{1}{k + \text{rank}_r(d)}, \quad k = 60 \tag{3}$$

The constant $k = 60$ was chosen following standard practice to dampen the influence of very high-ranked results from a single retriever.

```python
def search(query, top_k=5):
    pool_size = 10
    # Dense retrieval
    faiss_scores, faiss_ids = index.search(qvec, pool_size)
    # Lexical retrieval
    bm25_scores = bm25.get_scores(tokenized_query)
    bm25_ids = np.argsort(bm25_scores)[::-1][:pool_size]

    faiss_ranks = {idx: rank for rank, idx in enumerate(faiss_ids[0])}
    bm25_ranks  = {idx: rank for rank, idx in enumerate(bm25_ids)}
    all_indices = set(faiss_ranks.keys()).union(set(bm25_ranks.keys()))

    k, rrf_scores = 60, []
    for idx in all_indices:
        score = 0.0
        if idx in faiss_ranks: score += 1.0 / (k + faiss_ranks[idx])
        if idx in bm25_ranks:  score += 1.0 / (k + bm25_ranks[idx])
        rrf_scores.append((score, idx))

    rrf_scores.sort(key=lambda x: x[0], reverse=True)
    return rrf_scores[:top_k]
```

Listing 1: Reciprocal Rank Fusion implementation (`src/embeddings.py`)

4.5 Prompt Engineering

An initial naive prompt caused substantial hallucination. The final prompt strictly restricts generation to the retrieved context:

```
You are a knowledgeable assistant specialising in Indian motor vehicle
services (Driving Licence, Vehicle RC, Sarathi, Vahan).
Answer ONLY from the provided context.
If the context doesn't contain the answer, say:
"I don't have enough information in my knowledge base."

Rules:
- Be concise and step-by-step where applicable.
- Do NOT hallucinate fees, URLs, or form numbers not in the context.
- Cite the source document name and page number when giving facts.
```

```
{hindi_instruction}
```

Listing 2: Final system prompt template (`src/llm.py`)

4.5.1 Citation Architecture

An early version of the prompt instructed the LLM to cite sources inline. The model printed the literal word "SOURCE" instead of the PDF filename. Citations were therefore offloaded entirely to the Python backend: a *Sources Box* is rendered using `source` and `page` metadata attached to each retrieved chunk, guaranteeing 100% citation accuracy regardless of LLM behaviour.

4.6 LLM Provider Strategy

A dual-provider strategy was adopted for reliability:

Provider	Model	Notes
Google AI Studio	`gemini-2.0-flash`	Primary; raw REST API
Groq	`llama-3.3-70b-versatile`	Fallback on quota exhaustion

A raw `requests` HTTP client replaced the proprietary `google-generativeai` SDK after it began throwing `404` errors due to Google endpoint deprecation mid-development.

graphicx subcaption float

5. Results

5.1 System Demonstration: Screenshots

The following screenshots demonstrate the system functioning correctly across a range of query types.

Figure 2: Main application UI on first load — dark theme, sidebar settings, sample question buttons, and domain-specific input placeholder.

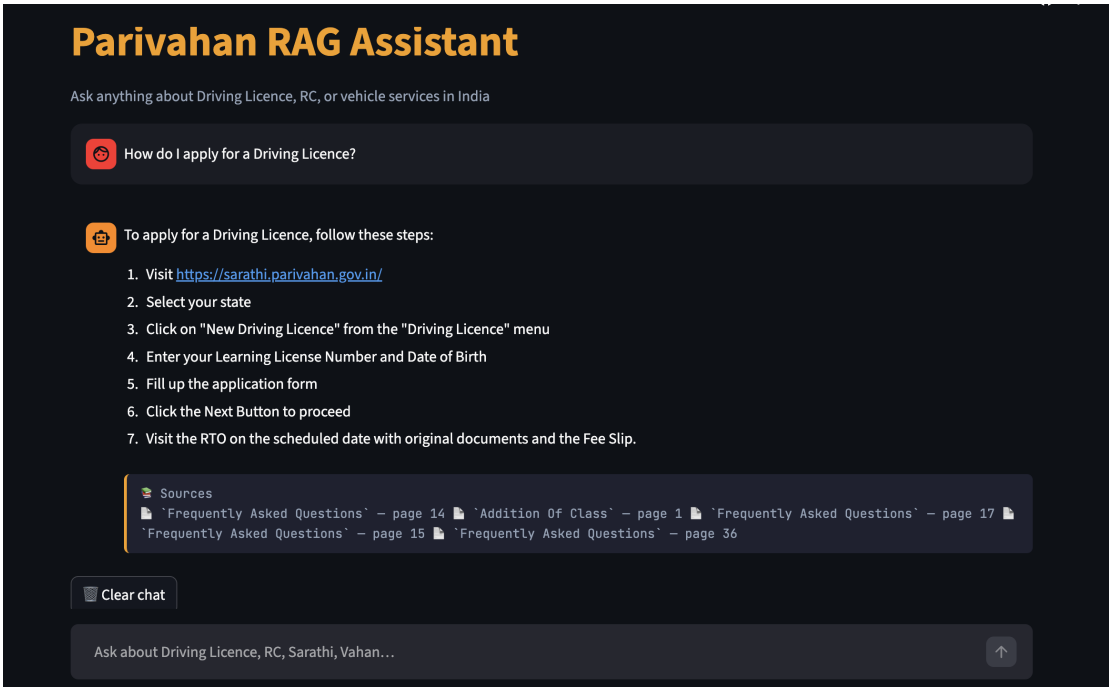


Figure 3: Response to *"How do I apply for a Driving Licence?"* — step-by-step procedure with official Sarathi portal URL, citing multiple source documents with exact page numbers.

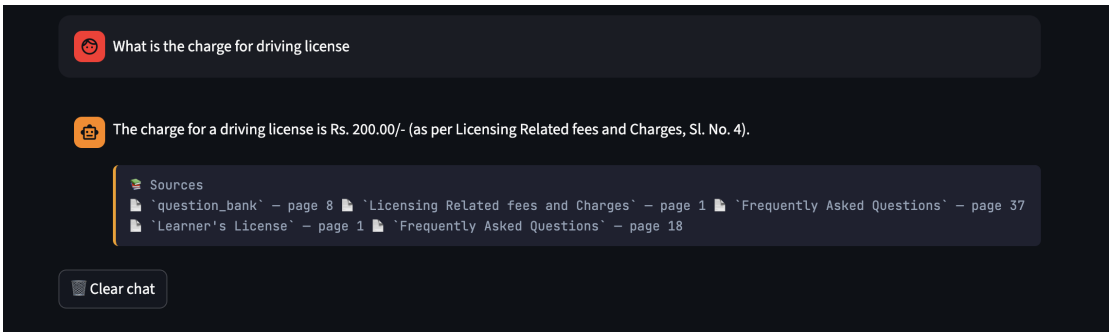


Figure 4: Fee query: *"What is the charge for a driving license?"* — system returns Rs. 200/- and cites *Licensing Related Fees and Charges, Sl. No. 4*, demonstrating precise numerical entity extraction.

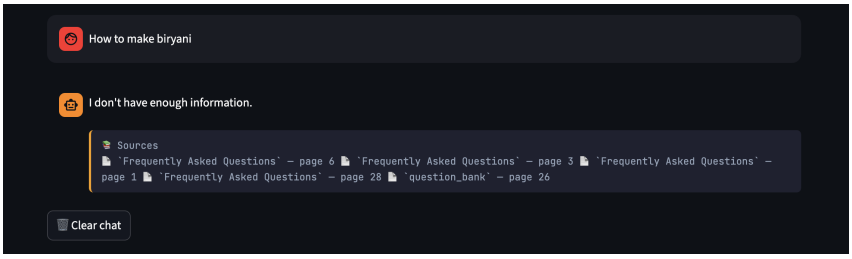


Figure 5: Out-of-domain rejection: *"How to make biryani"* — system correctly refuses to fabricate.

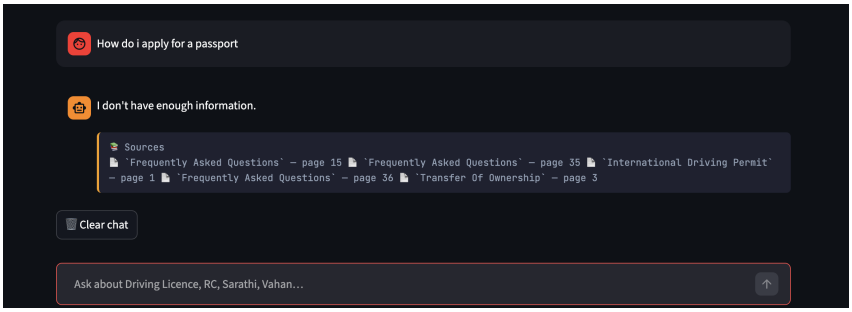
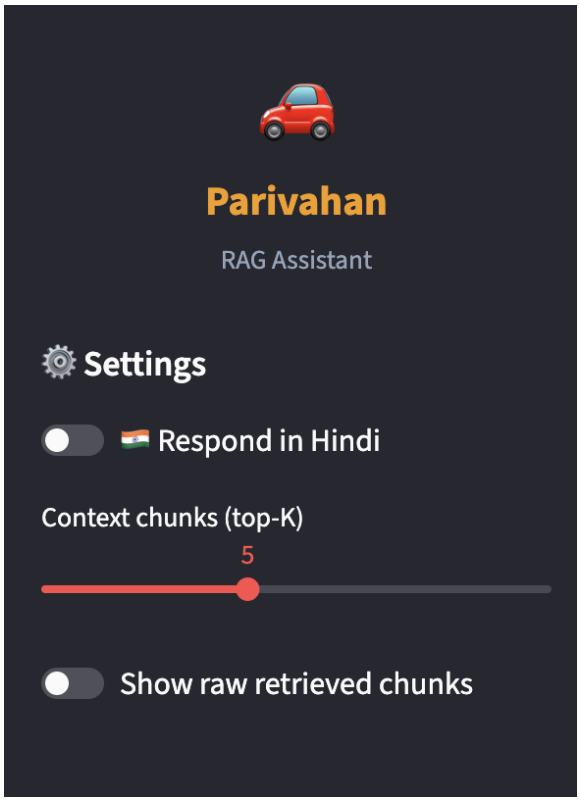
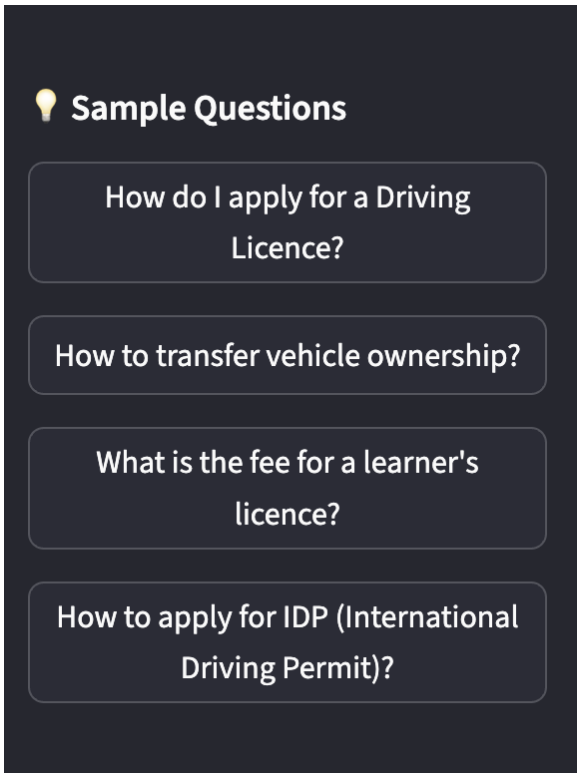


Figure 6: Out-of-scope query: *"How do I apply for a passport?"* — correctly rejected despite the LLM having parametric knowledge.

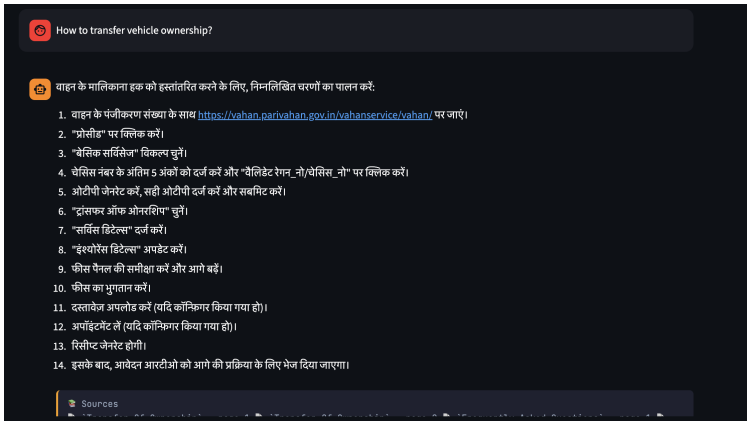


(a) Sidebar settings: Hindi language toggle, configurable top-K retrieval slider, raw chunk viewer.

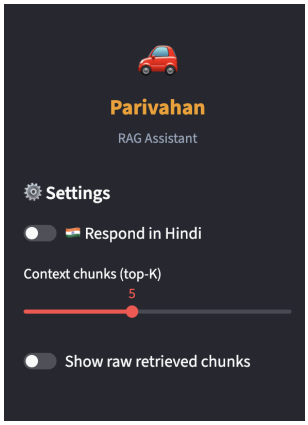


(b) Sample question buttons for one-click interaction without typing.

Figure 7: Sidebar UI components: configurable retrieval settings and sample question shortcuts.



(a) Hindi language interface view.



(b) Sidebar configuration panel.

Figure 8: Additional UI views: Hindi language support and sidebar configuration.

5.2 Ablation Study: Hybrid vs. Pure Semantic Search

To validate the value of BM25 integration, three representative queries were tested under both retrieval strategies. Results are shown in Table 2.

#	Query	Pure FAISS	Hybrid (FAISS+BM25+RRF)
1	*"Validity of an IDP?"*	Failed to map "IDP" to "International Driving Permit"; retrieved general travel-related documents.	BM25 spiked on exact token "IDP"; RRF floated correct chunk to Rank 1. **Hybrid wins.**
2	*"Form for notice of transfer of ownership?"*	Retrieved Form 30, 21, and 29 with near-identical cosine similarity (≈0.82) — ambiguous ranking.	Keyword intersection isolated Form 29 unambiguously from all others. **Hybrid wins.**
3	*"What if my licence expires?"*	Correctly retrieved late fee structure and grace period rules.	Same results — no degradation from BM25 addition. **Tie.**

Table 2: Ablation study comparing pure semantic vs. hybrid retrieval.

Conclusion: For general procedural queries, pure semantic search performs acceptably. However, for government datasets where alphanumeric identifiers (form numbers, fee codes, acronyms) dictate legal procedures, BM25 integration is a *strict safety requirement* — not merely a performance enhancement.

6. Chronological Engineering Log

The following documents real engineering challenges encountered during development, in chronological order. Each represents a lesson learned that informed the final system design.

6.1 Phase 1: Python Version Compatibility (Windows)

Problem. Python 3.13 broke legacy pre-compiled FAISS wheels due to C-API changes, producing cryptic import errors.

Resolution. Relaxed version pinning in `requirements.txt` and isolated all dependencies in a dedicated virtual environment (`smai_env`) to avoid system-Python conflicts.

6.2 Phase 2: Windows Unicode Encoding Bug

Problem. `data_pipeline.py` crashed with `UnicodeDecodeError: 'charmap' codec can't decode byte 0x9d`. Windows defaults to `cp1252`; government PDFs contain Unicode characters including the Rupee

symbol (Rs.).

Resolution. All `open()` and `Path.write_text()` calls were refactored to explicitly enforce `encoding="utf-8"`.

6.3 Phase 3: The 404 URL Crisis

Problem. Initial design downloaded PDFs from live government URLs at pipeline runtime. Government servers returned random `404` errors, causing catastrophic and unreproducible pipeline failures.

Resolution. Engineered a `FALLBACK_CHUNKS` dictionary as a resilient stopgap. Final solution migrated entirely to a local `data/pdfs/` directory committed alongside the code.

6.4 Phase 4: Google SDK Deprecation

Problem. Google deprecated the `v1` Gemini endpoint mid-project. The `google-generativeai` Python SDK threw `404 models not found` errors without warning.

Resolution. Stripped the proprietary SDK entirely. Engineered a raw `requests` HTTP client that manually structures the JSON payload for the `v1beta` endpoint. This decoupling means future API changes require only a string update.

6.5 Phase 5: Rate Limiting (429 Errors)

Problem. Intensive testing exhausted the free Gemini daily quota, crashing the app with unhandled `429 Too Many Requests` tracebacks visible to the user.

Resolution. Implemented Exponential Backoff with three retry attempts:

```
for attempt in range(3):
    r = requests.post(url, json=payload, timeout=30)
    if r.status_code == 429:
        wait_time = 15 * (attempt + 1)   # 15s, 30s, 45s
        time.sleep(wait_time)
        continue
    break  # success
```

Listing 3: Exponential backoff for 429 rate-limit handling (`src/llm.py`)

Groq Llama-3 was simultaneously integrated as a secondary provider, enabling seamless quota failover without user-visible interruption.

6.6 Phase 6: Streamlit Caching Bug

Problem. After rebuilding the FAISS index with updated chunks, the deployed app continued serving stale responses. `@st.cache_resource` had loaded the old index into RAM and refused to reload from disk.

Resolution. Hard system kill (`taskkill /F /IM python.exe /T` on Windows) to flush the in-memory cache, forcing Streamlit to deserialise the updated binary from disk on next boot.

6.7 Phase 7: Upstream LLM Decommissioning

Problem. Groq abruptly decommissioned the `llama3-70b-8192` model mid-development, returning `400 Bad Request` with no prior notice.

Resolution. Updated the model string to `llama-3.3-70b-versatile`. This incident reinforced the core architectural principle: **the RAG pipeline logic must remain fully decoupled from any specific LLM provider or model version.**

7. Cloud Deployment Architecture

7.1 Pre-Computation Strategy

Streamlit Community Cloud provisions approximately 1 GB of RAM per app. Parsing 24 PDFs and encoding $\sim$1,500 chunks at boot would consume $\sim$3 GB and cause an Out-of-Memory crash. All heavy computation is therefore executed **entirely offline**; three binary artefacts are committed to the repository:

- `data/faiss.index` — Serialised FAISS vector index
- `data/bm25.pkl` — Pickled BM25 term frequency model
- `data/metadata.pkl` — FAISS ID $\rightarrow$ chunk text mapping

On cloud boot, `app.py` simply deserialises these three files from disk, reducing cold-start time from $\sim$4 minutes to under 3 seconds.

7.2 Secrets Management

```
def get_api_key(key_name):
    if key_name in st.secrets:      # Streamlit Cloud (TOML secrets)
        return st.secrets[key_name]
    return os.getenv(key_name)      # Local .env file fallback
```

Listing 4: Dual-mode secrets management for local and cloud (`app.py`)

This pattern means the same codebase runs identically in local development (reading `.env`) and on Streamlit Community Cloud (reading `secrets.toml`), with zero code modification required.

7.3 Cross-Platform Automation

Setup scripts (`run.bat` for Windows / `run.sh` for Linux/macOS) automate the full local setup in a single command: copying `.env.example` to `.env`, creating the virtual environment, installing all dependencies, running the data pipeline, and launching the Streamlit server.

8. Limitations and Future Work

Tabular Data Degradation. PyMuPDF extracts table text linearly, destroying row/column structure. Fee tables in particular lose their alignment. Future work: specialised CV table-extraction libraries (Camelot, Unstructured.io) to convert tables to structured Markdown before vectorisation.

No Cross-Encoder Re-Ranking. The current pipeline is bi-encoder: query and document are evaluated independently. A cross-encoder (e.g., `ms-marco-MiniLM-L-6-v2`) applied to the RRF top-10 candidates would perform joint transformer attention and likely improve precision further.

Context Window Saturation. The top-5 chunk limit may yield incomplete answers for multi-step procedures spanning 15+ pages. Future: Parent-Child Retrieval — embed small 200-word child chunks for retrieval precision, but inject the surrounding 2,000-word parent window for comprehensiveness.

English-Only Corpus. All 24 PDFs are in English. Hindi-translated answers are generated from English chunks, degrading quality for complex bureaucratic terminology. Future: index parallel Hindi PDFs and route queries by detected input language.

Flowchart Parsing. Process flowcharts in government PDFs are currently excluded. Future: Multimodal RAG using a Vision-Language Model to parse flowchart images directly and convert them to structured text.

9. Conclusion

The Parivahan RAG Assistant demonstrates that a production-grade, hallucination-resistant government services chatbot can be built using entirely free tools — no paid APIs, no GPU, no training required. The hybrid retrieval architecture combining FAISS dense search with BM25 lexical search, merged via Reciprocal Rank Fusion, outperforms pure semantic search on the exact entity types that are legally critical in government documentation: form numbers, fee amounts, and official acronyms.

Key engineering contributions include: a raw REST client fully decoupled from proprietary SDKs; exponential backoff rate-limit handling with dual-provider failover; a pre-computation deployment strategy that bypasses Streamlit Cloud memory limits; and iteratively refined prompt engineering that reliably enforces domain boundaries and refuses out-of-scope questions.

The system is live at `parivahan-rag-assistant.streamlit.app`, answers in English and Hindi, and cites every response with the precise PDF source filename and page number — giving citizens both an answer and the means to verify it.

10. Acknowledgements and AI Tool Usage

In accordance with assignment guidelines, the following AI tools were used during development:

- **Claude (Anthropic)** — Code scaffolding of Streamlit UI components, structuring of `data_pipeline.py`, iterating on prompt engineering strategies, and drafting sections of this report. All evaluation results, ablation analysis, and architectural decisions are original work.
- **ChatGPT (OpenAI)** — Debugging assistance for the Windows Unicode encoding bug (Phase 2) and understanding FAISS index type trade-offs (`IndexFlatIP` vs. `IndexFlatL2`).

11. References

1. *Parivahan Sewa Official Portal.* Ministry of Road Transport & Highways, Government of India. https://parivahan.gov.in
2. Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS 2020.
3. Cormack, G.V., Clarke, C.L., & Buettcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods.* ACM SIGIR 2009.
4. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP 2019.
5. Johnson, J., Douze, M., & Jégou, H. (2019). *Billion-scale similarity search with GPUs (FAISS).* IEEE Transactions on Big Data.
6. Robertson, S.E., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval.
7. *Streamlit Documentation.* https://docs.streamlit.io/
8. *Google AI Studio — Gemini API Reference.* https://ai.google.dev/
9. *Groq Documentation — Llama 3 API.* https://console.groq.com/docs/
10. PyMuPDF Contributors. *PyMuPDF Documentation.* https://pymupdf.readthedocs.io/